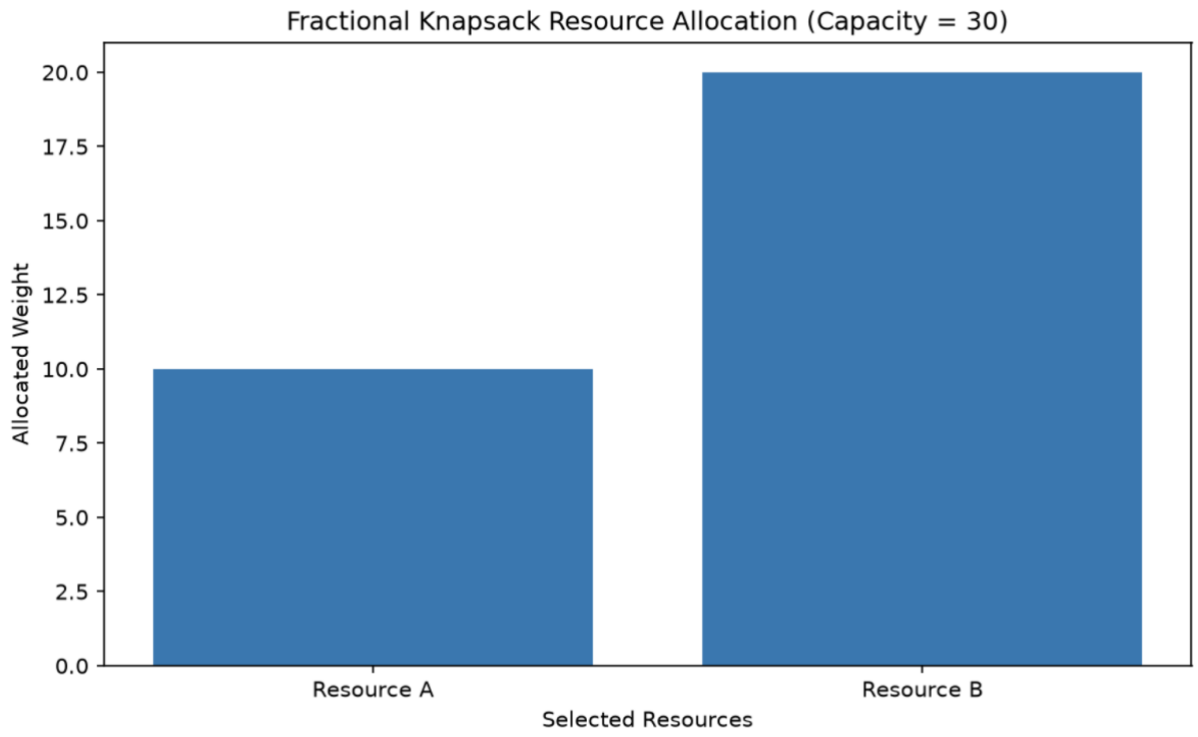
DAA LAB 3:

2. VISUALIZATIONS AND PERFORMANCE GRAPHS

1. Objective

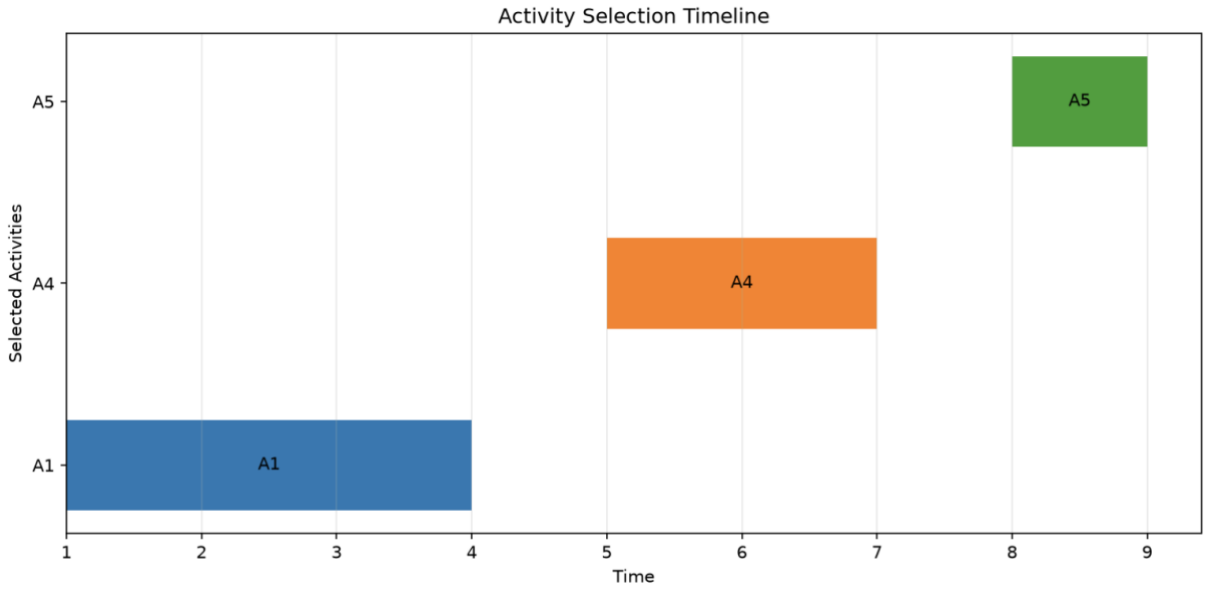
The objective is to visualize the results of three greedy algorithms and compare their performance through charts and graphs.

2. Fractional Knapsack — Resource Allocation Chart



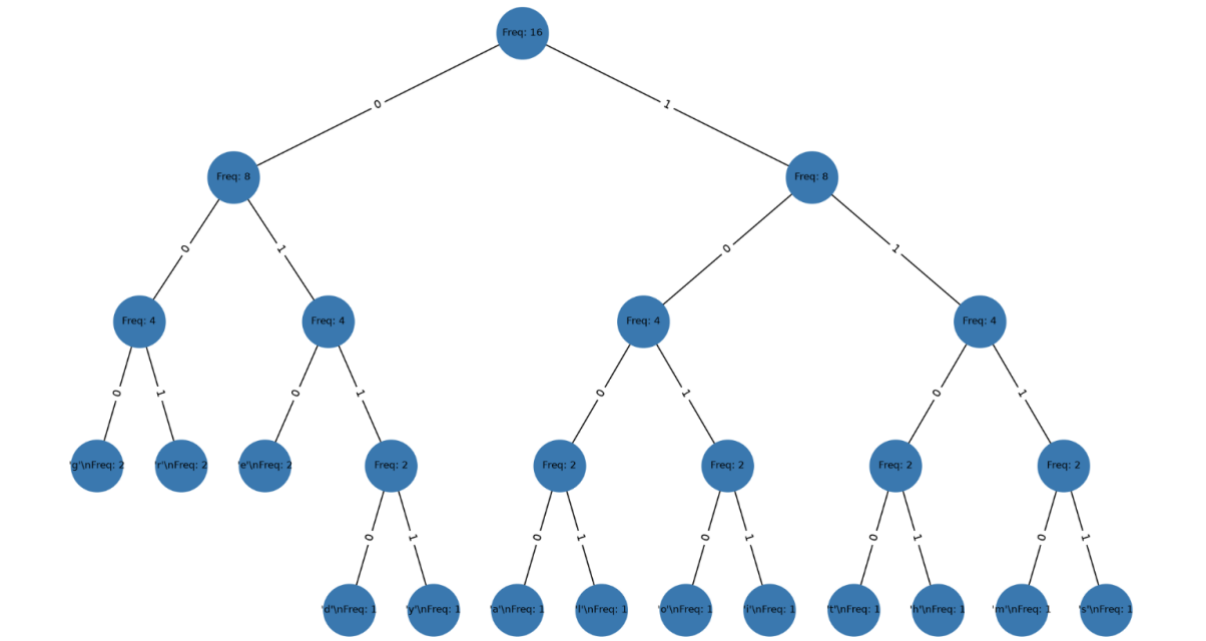
This chart shows how the available capacity is allocated among selected resources. Fractional Knapsack prioritizes items according to their value-to-weight ratios and allows partial selection of an item when the remaining capacity is insufficient for the entire item.

3. Activity Selection — Activity Timeline



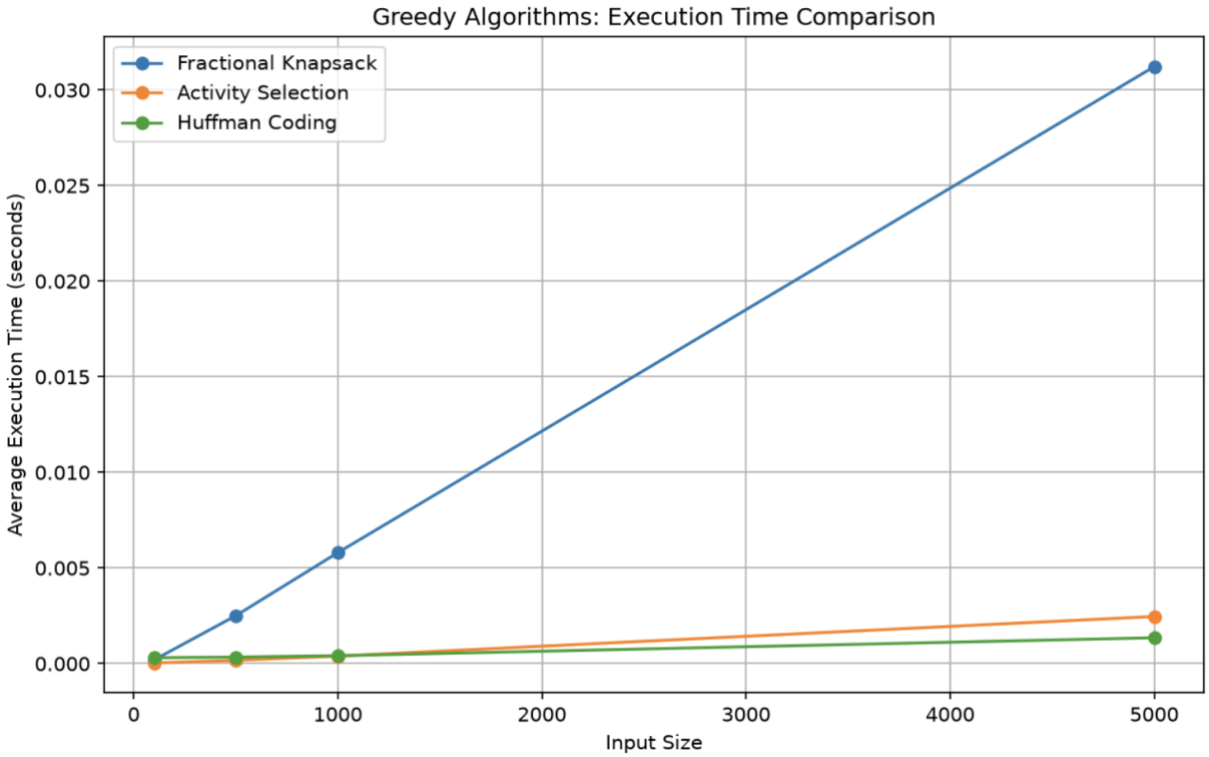
The timeline represents the activities selected by the Activity Selection algorithm. The algorithm sorts activities by finishing time and selects activities that do not overlap with previously selected activities.

4. Huffman Coding — Tree Representation



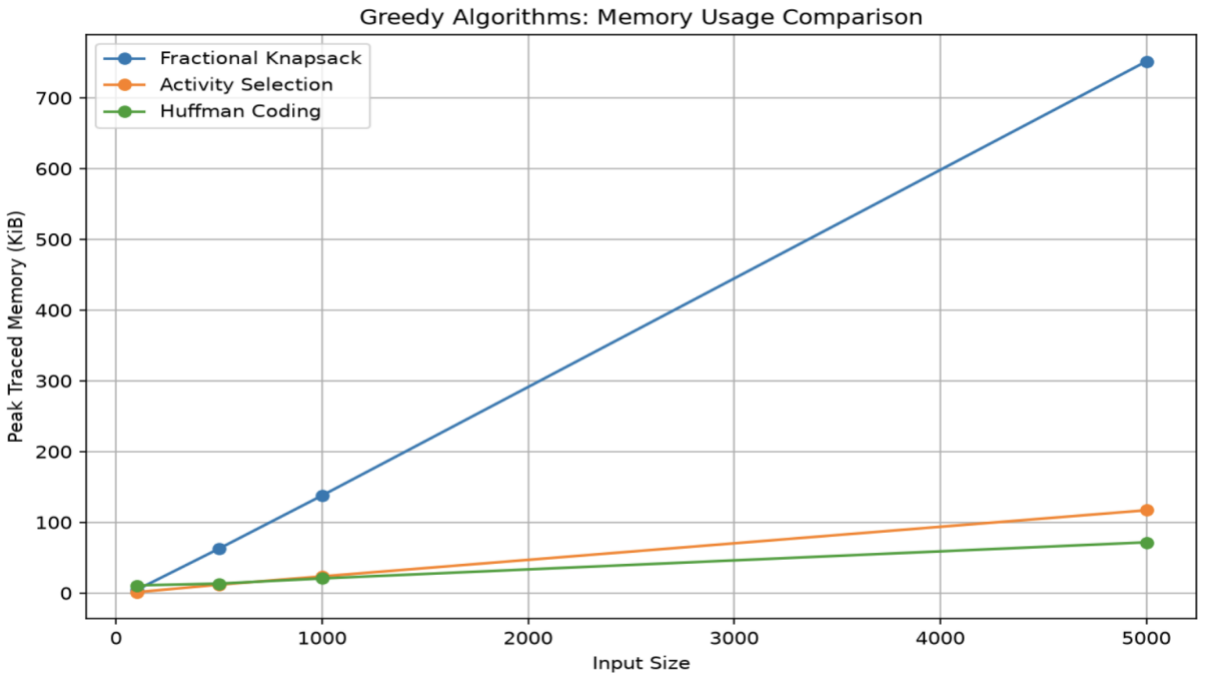
The Huffman tree represents the structure used to generate binary codes. Each leaf represents a character, and each internal node represents a combined frequency. The left and right edges correspond to binary digits 0 and 1.

5. Execution-Time Comparison



This graph compares the average execution time of Fractional Knapsack, Activity Selection, and Huffman Coding for input sizes of 100, 500, 1000, and 5000.

6. Memory-Usage Comparison



This graph compares peak traced memory usage for the three algorithms at different input sizes. Memory was measured using Python's `tracemalloc` module.

7. Observations

The resource allocation chart illustrates greedy selection based on item value and weight. The activity timeline demonstrates how compatible activities are selected using their finishing times. The Huffman tree illustrates how character frequencies determine binary codes.

The performance graphs show the measured execution time and peak traced memory. The observed values may vary with input data, system load, and runtime conditions.

8. Conclusion

The visualizations help explain the working of the three greedy algorithms and present their measured performance in a graphical format. Together, the charts and performance graphs support the theoretical and experimental analysis of the algorithms.